

Stealth Escape - Game Design Document

Game Summary

Stealth Escape is a 2D desktop stealth platformer built with Java and libGDX. The player must collect all items while avoiding guards, lasers, traps, and cameras, then reach the exit door. The win condition is escaping after collecting every item. The lose condition is being caught or hit by hazards.

Controls

- A / Left Arrow - Move left
- D / Right Arrow - Move right
- W / Up Arrow - Climb stairs up
- S / Down Arrow - Climb stairs down
- H - Hide
- P / Esc - Pause or resume
- Enter / Space - Continue from Game Over

Player

The player can move left and right, climb stairs, hide, collect items, activate switches, and respawn from checkpoints after being caught. The player must use stealth and timing to avoid detection.

Enemies and Objects

- Guards patrol the level and can detect or catch the player.
- Lasers are timed hazards that cause Game Over on contact.
- Cameras scan areas and detect the player.
- Traps are dangerous obstacles that defeat the player on contact.
- Items must be collected before the exit becomes available.
- Switches can control special level hazards.
- Checkpoints save the player's respawn position.

Levels

The game contains one polished level. The level includes platforms, walls, stairs, guards, lasers, cameras, traps, collectibles, checkpoints, and an exit door. The challenge comes from timing movement, hiding, and choosing a safe route.

Win and Lose Screen

When the player collects all items and reaches the exit, the Win screen is shown. When the player is caught by a guard, detected by a camera, hit by a laser, or touches a trap, the Game Over screen is shown and the player can continue from the latest checkpoint.

Art Style

The game uses simple 2D pixel-style sprites with a dark stealth-themed visual style. The UI is minimal and focuses on the item counter, menu, pause screen, game over screen, and win screen.

Design Patterns and SOLID

- **Single Responsibility:** PlayerController handles only input, Player handles only state.
- **Open/Closed:** New enemy types can extend the base without modifying existing classes.
- **State Pattern:** StateManager manages all screens (Menu, Game, Pause, GameOver, Win).
- **Separation of Concerns:** Assets, Collider, Camera are isolated utility classes.